Exercise 1 - solutions

Mattias Villani and Yulia Ryanova

Problem 1

Problem 1a) see Example 8.1 in Bayesian Learning - the prequel.

Problem 1b) see Problem 1 in Section 8.4 in Bayesian Learning - the prequel.

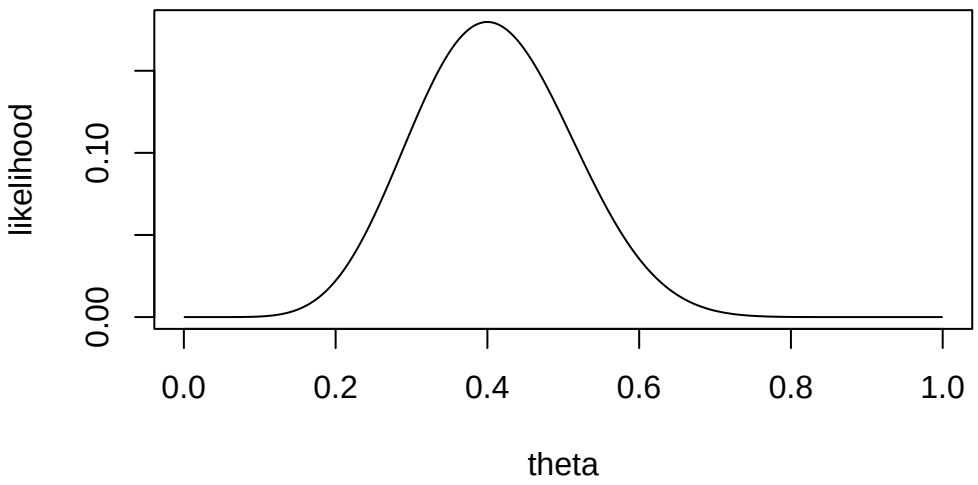
Problem 1c) see Problem 1 in Section 8.4 in Bayesian Learning - the prequel.

Problem 1d) The likelihood is

$$p(s|\theta) = \binom{n}{s}\theta^s(1-\theta)^f$$

which is the binomial distribution, so we can use the `dbinom` function in R:

```
n = 20
s = 8
thetaGrid = seq(0.001, 0.999, length = 1000) # grid of values where we plot the likelihood
like = dbinom(x = s, size =n, prob = thetaGrid)
plot(thetaGrid, like, type = "l", ylab = "likelihood", xlab = "theta")
```



The MLE is $\hat{\theta} = s/n$ with asymptotic distribution from the general asymptotic distribution of the MLE

$$\hat{\theta} \overset{\text{approx}}{\sim} N(\theta, \mathcal{J}_n^{-1}(\hat{\theta}))$$

where from problem 1b), the observed information is

$$\mathcal{J}_n^{-1}(\hat{\theta}) = \frac{n}{p(1-p)}$$

so

$$\hat{\theta} \overset{\text{approx}}{\sim} N\Big(\theta, \frac{\hat{\theta}(1-\hat{\theta})}{n}\Big)$$

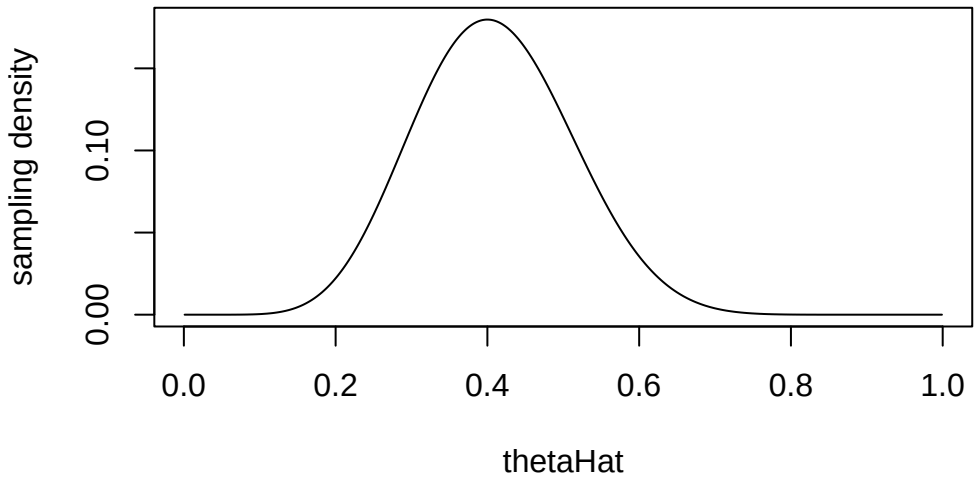
Note that this is a *sampling distribution* of the MLE, not the likelihood. In order to plot this we need to know the true value, which is of course not known. We may choose to set $\theta = \hat{\theta} = 8/20 = 0.4$ and plot the normal approximation

$$N\Big(\hat{\theta}, \frac{\hat{\theta}(1-\hat{\theta})}{n}\Big) = N\Big(0.4, \frac{0.4 \cdot 0.6}{20}\Big) = N(0.4, 0.12)$$

where 0.12 is a variance here, not a standard deviation.

Here is the plot

```
n = 20
s = 8
thetaHat = s/n # This is the MLE
thetaHatGrid = seq(0.001, 0.999, length = 1000) # grid of values
sampdist = dnorm(thetaHatGrid, mean = thetaHat, sd = sqrt(thetaHat*(1-thetaHat)/n))
plot(thetaGrid, like, type = "l", ylab = "sampling density", xlab = "thetaHat")
```


sampling density
0.00
0.10
0.0
0.2
0.4
0.6
0.8
1.0
thetaHat

Problem 2

MAXIMUM LIKELIHOOD

Let $X_1, X_2, \ldots, X_n$ be independent random variables with density function

$$f(x) = \alpha x^{\alpha-1} \quad \text{for } 0 < x < 1,$$

and $f(x) = 0$ otherwise. The parameter α is positive.

(a) *Points: 6p.* Derive the maximum likelihood estimator of α.

Solution:

The log-likelihood is

$$\ell(\alpha) = \sum_{i=1}^{n} \log f(x_i|\alpha),$$

where

$$f(x|\alpha) = \log \alpha + (\alpha - 1) \log x$$

so the log-likelihood is

$$\ell(\alpha) = \sum_{i=1}^{n} \log f(x_i|\alpha) = \sum_{i=1}^{n} (\log \alpha + (\alpha - 1) \log x_i) = n \log \alpha + (\alpha - 1) \sum_{i=1}^{n} \log x_i$$

We find the MLE by setting the derivative of the log-likelihood function to zero and solve for α

$$\ell'(\alpha) = \frac{n}{\alpha} + \sum_{i=1}^{n} \log x_i = 0$$

which gives the solution

$$\hat{\alpha} = -\frac{n}{\sum_{i=1}^{n} \log x_i}.$$

Note that is seems that the estimate is negative (because of the minus sign), but it is actually positive since each observation $0 < x_i < 1$ and therefore $\log x_i < 0$ for all x_i. To verify that this indeed a maximum we check that the second derivative of $\ell(\alpha)$ is negative at $\hat{\alpha}$. The second derivative is

$$\ell''(\alpha) = \frac{\mathrm{d}}{\mathrm{d}\alpha} \ell'(\alpha) = \frac{\mathrm{d}}{\mathrm{d}\alpha} \left(\frac{n}{\alpha} + \sum_{i=1}^{n} \log x_i \right) = -\frac{n}{\alpha^2} < 0$$

so this indeed a maximum.

(b) *Points*: *4p.* Determine the usual large sample approximation of the sampling distribution of the maximum likelihood estimator $\hat{\alpha}$.

Solution:

From the Formula sheet we have the normal approximation of the sampling distribution of the MLE

$$\hat{\alpha}(X_1, \ldots, X_n) \overset{\text{approx}}{\sim} N\left(\alpha, I_n^{-1}(\alpha)\right)$$

where $I_n(\theta)$ is the Fisher information

$$I_n(\alpha) = \mathbb{E}\left(-\ell''(\alpha)\right) = \mathbb{E}\left(\frac{n}{\alpha^2}\right) = \frac{n}{\alpha^2},$$

since $\frac{n}{\alpha^2}$ is a constant (with respect to the data $X_1, \ldots, X_n$) so the expectation is just this constant. Hence we have the large sample approximation

$$\hat{\alpha}(X_1, \ldots, X_n) \overset{\text{approx}}{\sim} N\left(\alpha, \frac{\alpha^2}{n}\right)$$

Problem 3

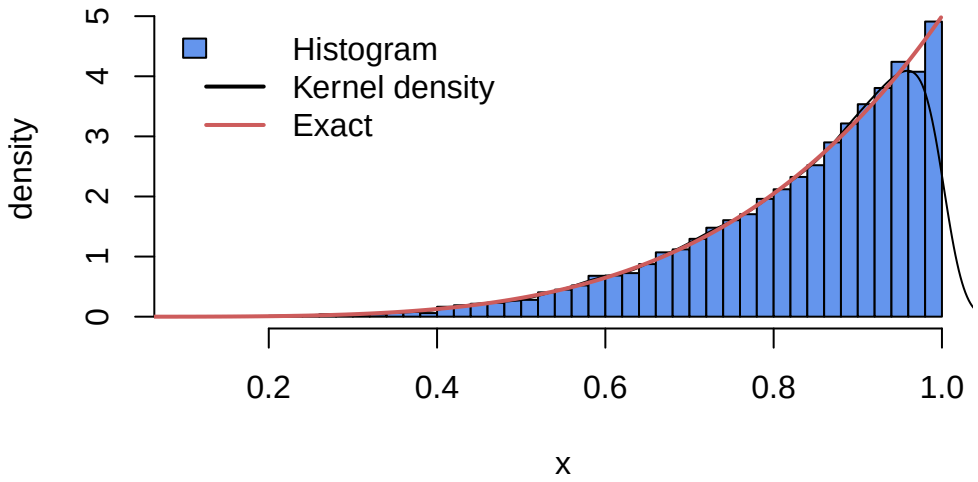
Problem 3a)

The CDF $F(x)$ is the anti-derivative of the PDF $f(x) = 5x^4$ hence $F(x) = x^5$. The inverse CDF method solves the following equation for $u \sim Unif(0, 1)$

$$F(x) = x^5 = u$$

which has solution $x = u^{1/5}$. So we can simulate X from $f(x) = 5x^4$ by simulating a uniform random number u (using `runif` in R) and just compute the inverse CDF value $x = u^{1/5}$. Here is the code, where I simulate 10000 draws and plot a histogram and kernel density estimate. I also overlay the true density, to check that my code is doing the right thing.

```
nSim = 10000
u = runif(n = nSim) # a whole vector of Unif(0,1) numbers
x = u^(1/5)         # a whole vector of draws from f(x)
hist(x, 50, freq = F, xlab = "x", ylab = "density", col = "cornflowerblue", main = "")
kde = density(x) # this can go outside the support (0,1)
lines(kde$x, kde$y)
xgrid = seq(0.001, 0.999, length = 1000)
exactdensity = 5*xgrid^4
lines(xgrid, exactdensity, col ="indianred", lw = 2)
legend("topleft",
       legend = c("Histogram", "Kernel density", "Exact"),
       fill = c("cornflowerblue", NA, NA),
       border = c("black", NA, NA),
       lty = c(NA, 1, 1),
       lwd = c(NA, 2, 2),
       col = c(NA, "black", "indianred"),
       bty = "n")
```



Problem 3b) We already have draws from the density $f(x)$ from Problem 3a). The Monte Carlo estimate is then the mean of the function values $\exp(x^{(i)})$ where $x^{(i)}$ is the ith draw. Let μ denote the sought mean $\mathbb{E}(\exp(X))$ the Monte Carlo estimator is then

$$\hat{\mu} = \frac{1}{n_{sim}} \sum_{i=1}^{n_{sim}} \exp(x^{(i)}),$$

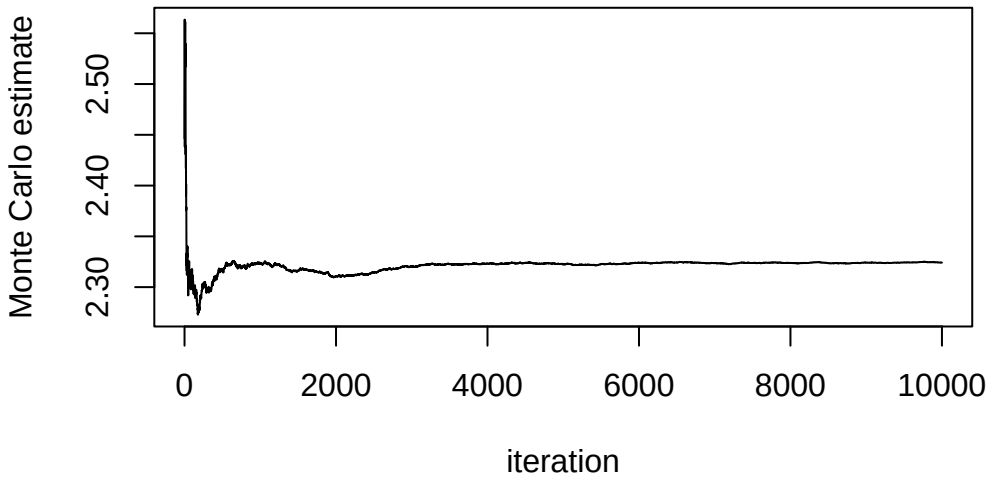
where n_{sim} is the number of Monte Carlo draws. This is the whole code (note that R is vectorized, so exp(x) where x is a vector that computes the `exp()` function for all elements in x).

```
mean(exp(x))
```

```
[1] 2.324162
```

We can check if the Monte Carlo estimate seems to have converged (in probability) by plotting the estimate based on a single draw, then based on two draws, then based on three draws, and so on, all the way to the estimate based on all 10000 draws. These estimates based on more and more draws should "settle down". This is seen in the plot below (there are smarter ways to compute this that avoids the loop, but this is reasonable fast for this simple problem).

```
function_values = exp(x) # this is a vector with exp(x) values
mc_estimates = rep(0,nSim)
for (i in 1:nSim){
  mc_estimates[i] = mean(exp(x[1:i]))
}
plot(1:nSim, mc_estimates, type = "l", ylab = "Monte Carlo estimate", xlab = "iteration")
```



Problem 3c) The numerical standard error of the Monte Carlo estimate is

$$\sqrt{\frac{\mathbb{V}(\exp(X))}{n_{sim}}}$$

where $X \sim f(x)$ the density given in the problem. Here $n_{n_{sim}} = 10000$ is the number of Monte Carlo draws. The variance in the numerator can be estimated by the usual sample variance of the function values. Here we go:

```
varianceFunctionValues = var(exp(x))
se = sqrt(varianceFunctionValues/nSim)
message("The Monte Carlo standard error is ", se)
```

```
The Monte Carlo standard error is 0.00300695727968979
```

This standard error describes the "usual" deviation from the true value that you can expect to get in simulation.

Problem 3d) The central limit theorem applies here since the Monte Carlo estimator is a sample mean. Denote the expectation that we are trying to estimate by $\mu = \mathbb{E}(\exp(X))$ and let the Monte Carlo estimator be denoted

$$\hat{\mu} = \frac{1}{n_{sim}} \sum_{i=1}^{n_{sim}} \exp(X^{(i)})$$

The central limit theorem then says that

$$\hat{\mu} \overset{approx}{\sim} N\Big(\mu, \frac{\mathbb{V}(\exp(X))}{n_{sim}}\Big)$$

and the variance of this distribution is the square of the standard error computed above.

Problem 4

Problem 4a) The bounding constant must be a constant that is at least as large as the largest density value. Let us therefore find the maximum of the density. If $\alpha > 1$ then the mode of the density is at $x = 1$ (since the density is monotonically increasing in x) so the highest density value is $f(1) = \alpha(1)^{\alpha-1} = \alpha$. So $M = \alpha$ is a suitable bounding constant here.

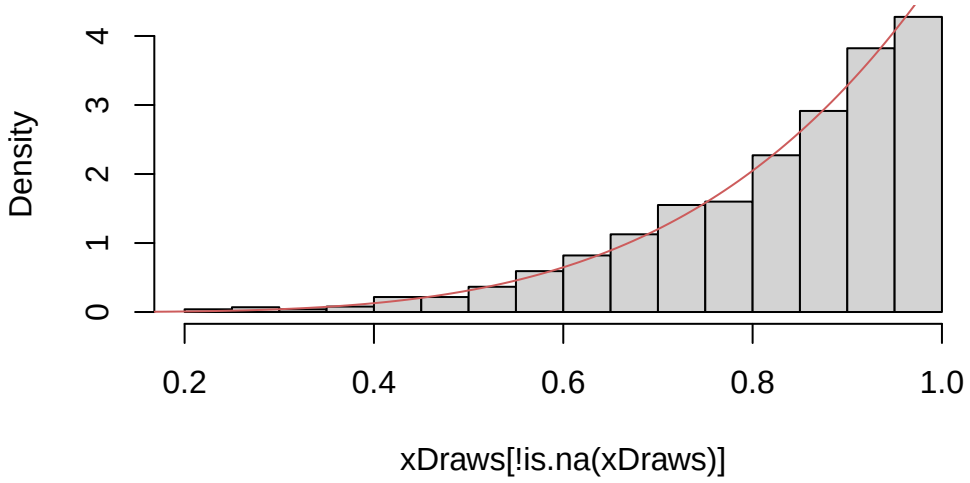
Problem 4b)

```
alpha = 5
nSim = 10000
xStar = runif(nSim) # these are the proposal draws from q()
M = alpha           # bounding constant

# This is the target density that we want to simulate from
pDensity <- function(x, alpha = 5){
  alpha*x^(alpha-1)
}

# This is the proposal density.
# A uniform distribution with density equal to 1 for all x
qDensity <- function(x){
  return(1)
}

xDraws = rep(0, nSim) # storage for the draws
for (i in 1:nSim){
  xStar = runif(1) # this is the proposal/candidate draw
  accProb = pDensity(xStar) / (M * qDensity(xStar)) # p(xStar)/(M*q(xStar))
  u = runif(1)
  if (u < accProb){
    xDraws[i] = xStar # draw is accepted!
  }else{
    xDraws[i] = NA # draw is rejected, insert NA
  }
}
nRejected = sum(is.na(xDraws))
# !is.na(xDraws) picks out all values that are not NA
hist(xDraws[!is.na(xDraws)], freq = F, main = "")
xGrid = seq(0.001,0.999, length = 1000)
lines(xGrid, pDensity(xGrid), col = "indianred")
```



Problem 4c) It is not very efficient. The uniform distribution looks nothing like the target density, so we have to choose a large bounding constant $M = \alpha = 5$ here. We therefore get many rejections:

```
message(nRejected, " draws were rejected out of the nSim = ",nSim," draws")
```

```
7975 draws were rejected out of the nSim = 10000 draws
```

In fact, the expected proportion of rejected draws is $1/M = 1/5 = 0.2$. So only 20% can be expected to the accepted. It would have been better with a proposal distribution that is closer to the target density, that is, similar shape.

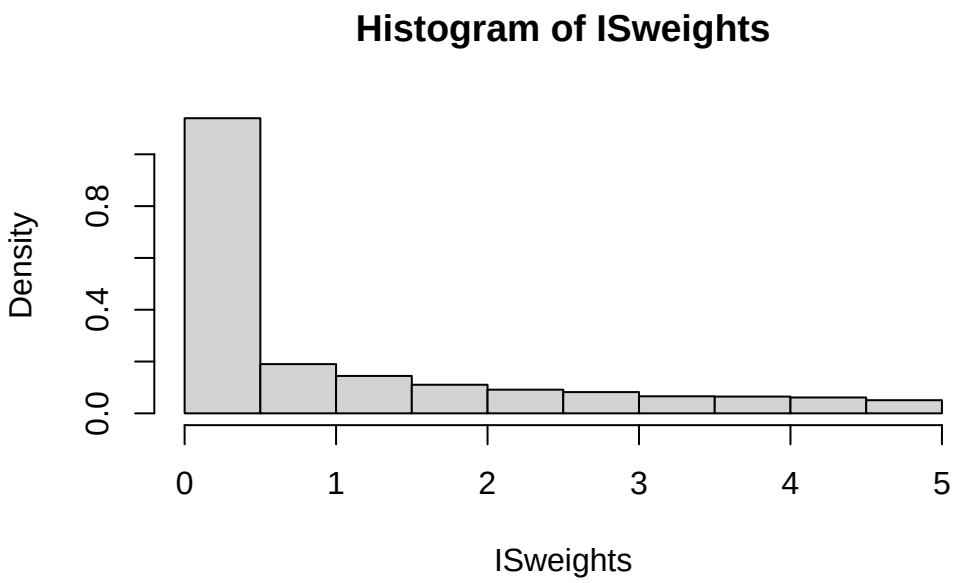
Problem 4d) Here is the importance sampling estimate

```
xDraws = runif(nSim)
ISweights = pDensity(xDraws)/qDensity(xDraws)
mean(ISweights * exp(xDraws))
```

```
[1] 2.337725
```

The weights should ideally be around one. In particular, it is bad (large variance of the IS estimator) if some weight are much larger than others. Here is a histogram of the weights

```
hist(ISweights, freq = F)
```



Or the log weights (easier to see when some weights are really large. A log weight around zero is ideal)

```
hist(log(ISweights), freq = F)
```

